

Introduction

- Operating System: Program that acts as an intermediary between a computer user and computer hardware
- First Computers: No OS, directly interact and change hardware, minimal overhead, but inefficient and not portable
- Mainframes: Accept programs in physical form, only support batch processing, very costly
- Batch OS: Execute user program / job one at a time, still interacts with hardware directly, with additional information for OS such as required resources and job specification
- Batch processing is inefficient: CPU idle when performing I/O
- Can use multiprogramming to load multiple jobs at once to pipeline, overlapping computation and I/O
- Time-Sharing OS: Allow multiple users to interact with machine using terminals / teletypes, user job scheduling gives illusion of concurrency, supports memory management
- Virtualisation: each program executes as if it has all resources to itself
- Personal OS: Machine dedicated to user, not timeshared

Motivation

- Abstraction: Hide low level details and present common high level functionality to user
- Large variation in hardware configurations and low-level details which user might not be interested in
- Resource Allocator: OS manages all resources and arbitrates conflicting requests for efficient and fair use
- Control Program: OS controls execution to prevent errors and improper use, providing security, isolation and protection
- Improves Program Portability and Efficiency

OS Structure

- OS is software that runs in kernel mode with direct access to hardware resources
- Other software executes in user mode with limited access to hardware resources
- User programs call OS using system call interface
- Kernel is a program with special features to deal with hardware and provide the syscall interface, with special code for interrupt handlers and device drivers
- Kernel code cannot use syscalls, normal libraries, or I/O
- Programming Language for OS: Historically in assembly, but now in higher level languages like C/C++, hardware architecture dependent
- Code Organisation: Machine independent / dependent HLL / assembly code

OS Structures

- Monolithic: Kernel is one big special program, various services and components are integral part
- Good performance but complicated internal structure with highly coupled components
- Microkernel: Small and clean, only provides basic and essential facilities such as IPC, address space and thread management
- Higher level services are built on top, and run as server process outside of OS, using IPC to communicate
- More robust and extendible, with better isolation and protection, but lower performance

- Layered Systems: Generalisation of Monolithic System, organising components into hierarchy of layers with lowest being hardware and highest being user interface
- Client-Server Model: Variation of microkernel, with two classes of processes; Client process request service from server process, which is built on top of microkernel, can even be on different machines

Virtual Machines

- Allows us to run multiples OSES on the same hardware at the same time, as well as debug / monitor them
- Virtual Machine: Software emulation / virtualisation of hardware, created and managed by hypervisor / Virtual Machine Monitor
- Type 1 Hypervisor: Provides individual virtual machines to guest OSES, bare metal hypervisor directly on hardware
- Type 2 Hypervisor: Runs in host OS, guest OS runs inside virtual machine

Process Abstraction

Memory Context: Function Call

- Important Steps
 - Setup parameters
 - Transfer control to callee
 - Setup local variables
 - Store result if applicable
 - Return to caller
- Control Flow issues: Need to jump to function body, and resume when function call is done; minimally need PC of caller
- Data Storage issues: Need to pass parameters to the function, and capture return result, function might also have local variables
- Need a new region of memory that is dynamically used by function invocations
- Stack Memory: New memory region to store information for a function invocation, described by a stack frame
- Stack Frame contains ra / PC, arguments / parameters, storage for local variables, other info
- Stack Pointer: Points to first unused location, usually in a specialised register
- Function Call Convention (Different ways to setup stack frame)
 - Prepare to make call: Caller passes parameters with registers or stack, then saves return PC on stack
 - Transfer control to callee: Callee saves old stack pointer, allocates space for local variables, and adjusts stack pointer to new stack top
 - On returning from function call: Callee places return result in register, and restores saved stack pointer
 - Transfer control back to caller using saved PC: Caller utilises return result, and continues execution
- Other information: Frame pointer, saved registers
- Frame Pointer: Facilitates access of various stack frame items, might be dedicated register, points to fixed location in a stack frame so that other items can be accessed as a displacement from it, each stack frame has one
- Saved registers: Use memory to hold GPR values when exhausted, so it can be used and restored, known as register spilling
- Function can spill registers it intends to use before function starts, to be restored at end of function
- Updated Steps
 - Prepare to make call: Caller passes parameters with registers or stack, then saves return PC on stack
 - Transfer control to callee: Callee saves registers used by callee, old stack pointer, old frame pointer, allocates space for local variables on stack, and adjusts stack pointer to new stack top, adjusts frame pointer
 - On returning from function call: Callee restores saved registers, FP, SP, places return result in register

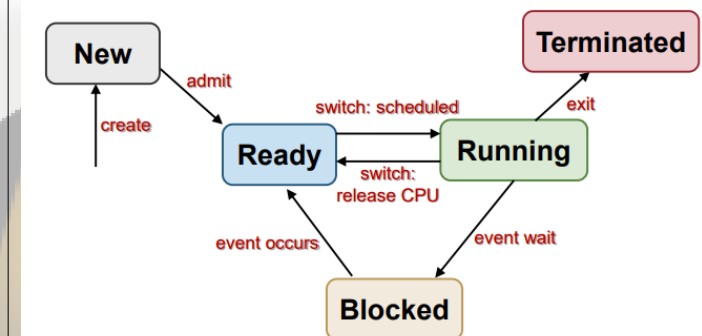
- Transfer control back to caller using saved PC: Caller utilises return result, and continues execution
- Stack Frame Illustration:
 - Free Memory
 - Local Variables
 - Parameters
 - Saved Registers
 - Saved SP (Top of previous stack frame)
 - Saved FP (Base address of previous stack frame)
 - Return PC
 - Previous stack frame

Dynamically Allocated Memory

- Memory space acquired during execution time
- Blocks allocated at runtime cannot be placed in data region
- Blocks with no definite deallocation timing cannot be placed in stack region
- Create separate heap memory for dynamic allocation
- Heap memory is harder to manage due to variable size and allocation/deallocation, and adverse scenarios can be constructed
- Text memory: For instructions
- Data memory: For global variables

Process Management

- To switch between programs, OS requires information regarding execution of current program to be stored, and replaced with new information for next program to run
- Need abstraction to describe a running program, known as process
- Process Abstraction: Process / Task / Job is a dynamic abstraction for executing program, includes memory, hardware, OS context
- Process ID: Used to distinguish processes from each other, unique
- Process State: Processes can be either running, or not running, but can also be in a ready state
- Process Model: Set of states and transitions
- 5-Stage Model
 - New process created, undergoes initialisation
 - Ready: Process is waiting to be run
 - Running: Process is being executed on CPU
 - Blocked: Process waiting / sleeping for event, cannot execute until event is available e.g. syscall, I/O
 - Terminated: Process finished execution, might need OS cleanup
- Queuing Model: OS has ready and blocked queue with processes inside, awaiting execution



Process Table and Control Block

- Process Control Block (PCB) or Process Table Entry: Entire execution context for a process
- Kernel maintains PCB for all processes in a table
- Issues: Scalability and Efficiency

System Calls

- API to OS, not same as normal function call, must change from user mode to kernel mode
- API is dependent on OS
- In C/C++, system calls can be invoked almost directly with library version acting as function wrapper
- Some library functions are more user friendly and serve as function adapters, not one to one
- General System Call Mechanism
 - User program invokes library call using normal function call mechanism
 - Library call (usually in assembly) places system call number in designated location such as register
 - Library call executes special instruction (TRAP) to switch from user to kernel mode, saves CPU state
 - Appropriate system call handler is determined, using system call number as index, handled by dispatcher
 - System call handler is executed
 - System call handler ends, restores CPU state, returns to library call, switch back to user mode
 - Library call returns to user program via normal function return mechanism

Exception and Interrupt

- Exceptions can be caused by executing machine level instructions, e.g. Arithmetic / Memory Access errors
- Exceptions are synchronous, occurring due to execution
- Exception handler is executed, similar to a forced function call
- Interrupts are external events that can interrupt the execution of a program, usually hardware related
- Interrupts are asynchronous, independent of program execution
- Program execution is suspended, and an interrupt handler must be executed
- Control transfer to handler routine automatically, before being returned to

Process Abstraction in Unix

- Processes are identified by integer Process IDs
- Information: Process State (Running, Sleeping, Stopped, Zombie), Parent PID, Cumulative CPU time
- Process creation: Using fork()
- Creates a new child process which is duplicate of current executable image, data is a copy of parent (copy on write)
- Child only differs in PID, PPID, and fork() return value (PID of new process for parent, 0 for child)
- Both continue execution after fork()
- For a different program, can use exec() variations instead
- `exec1(path, arg0, arg1, NULL)`: Replace current executing process image with a new one, code replacement, PID and other information intact, pass in path and arguments
- Master process: `init`, created in kernel at boot up, has PID 1, forms process tree
- Use `exit()` to end execution of process, 0 is normal exit
- On finish, most system resources used are released such as file descriptors, while basic process resources are not like PID and status as well as process accounting info since process table entry might still be needed
- Parent Child Synchronisation: Parent process waits for immediate child process to terminate using `wait(int *status)`

- Wait is blocking until at least one child terminates, and cleans up remainder of child system resources, has other variations as well
- On process exit, zombie is created since not all process info is deleted, `wait()` cleans it up
- Orphan: Parent terminates before child process: `init` becomes psuedo parent, then child termination sends signal to `init` which cleans it up with `wait()`
- Zombie: Child process terminates before parent, who did not `wait()`: Child becomes a zombie, and can fill up process table

Implementation of fork()

- Makes an almost exact copy of parent process
 - Create address space of child process
 - Allocate new PID
 - Create kernel process data structures like entry in process table
 - Copy kernel environment of parent process such as priority
 - Initialise child process context like PID, PPID, CPU time
 - Copy memory regions from parent such as Program, Data, Stack, but this is expensive
 - Acquire shared resources like open files, current working directory
 - Initialise hardware context for child processes by copying registers from parent process or others
 - Add child process to scheduler queue to wait to be run
- Memory Copy is expensive, as potentially the whole memory space needs to be copied
- However, child process will not access the whole memory range immediately, and shared version can be used if read only
- Copy on Write: Only duplicate a memory location when written to
- Memory is stored in memory pages and managed on a page level instead

Process Scheduling

Concurrent Execution

- Concurrent Processes: Multiple processes progress in execution at the same time, could be virtual or physical
- Timeslicing: Using 1 CPU, interleave instructions from both processes
- Multitasking needs to change context between processes, which incurs overhead
- Multiprocessor: Timeslicing on multiple CPUs at the same time
- Scheduling: How to organise processes to run
- Scheduler: Part of OS which makes scheduling decision
- Scheduling Algorithm: Algorithm used by scheduler
- Process Behaviour: Each process requires different amounts of CPU time for CPU-Activity and IO-Activity, can be either CPU/IO-Bound
- Processing Environment: Batch Processing (No interaction required, no need to be responsive), Interactive (User actively interacts with system, needs to be responsive and consistent in responses), Real Time Processing (Have deadline to meet, usually periodic)
- Scheduling Algorithms: Influenced by processing environment, common criteria are Fairness (Each get fair share of CPU time, no starvation), Utilisation (All parts of computing system should be utilised)
- Scheduling Policies: Non-Preemptive / Cooperative (Process stays running until it blocks or gives up CPU), Preemptive (Process given fixed time quota to run, and gets suspended after quota is up)
- Scheduling Process
 - Scheduler triggered, OS takes over
 - If context switching is needed, context of current running process is saved and placed on blocked or ready queue
 - Pick a suitable process to run based on scheduling algorithm
 - Setup context for this process, and let it run

Scheduling for Batch Processing

- No user interaction, non-preemptive scheduling is predominant
- Criteria: Turnaround time (Total time taken, considers waiting time), Throughput (Rate of task completion), CPU Utilisation (Percentage of time CPU spends working)
- First Come First Served (FCFS)
 - Tasks stored in queue based on arrival time
 - Earliest one is run until done or blocked
 - Blocked task is removed from queue, and placed at back when ready again
 - Guaranteed to have no starvation, each task will eventually be run
 - Waiting time can be reduced with simple reordering by having longer tasks later
 - Convoy effect: CPU-Bound task run first, before IO-Bound tasks, which waste time waiting before being immediately blocked
- Shortest Job First (SJF)
 - Select task with the smallest total CPU time
 - However, total CPU time needs to be known in advance which might have to be guessed
 - Minimises average waiting time, but might cause starvation due to bias towards short jobs
 - Prediction is done using previous CPU-Bound phases
 - Exponential Average: $P_{n+1} = \alpha \cdot A_n + (1 - \alpha) \cdot P_n$, using actual and past CPU time consumption
- Shortest Remaining Time (SRT)
 - Preemptive, use shortest remaining time
 - New job with shorter remaining time can skip queue, old jobs might get pushed back

Scheduling for Interactive Systems

- Criteria: Response Time, Predictability (Variation in response time)
- Preemptive scheduling algorithms are used to ensure good response time, and scheduler needs to run periodically
- Timer interrupt handler is used to ensure scheduler can run periodically without being stopped by other programs
- Interval of Timer Interrupt (ITI): Typically a few ms, OS scheduler invoked on every timer interrupt
- Time Quantum: Execution duration given to a process, multiple of ITI, large range of values, can be variable
- Round Robin
 - Tasks stored in FIFO queue
 - Task is picked up and run until time slice / quantum is elapsed or if it gives up early
 - Task is moved to back of queue to wait for another run, or in another queue if blocked
 - Preemptive version of FCFS
 - Response Time Guarantee: Given n tasks and quantum q , time before a task gets CPU is bounded by $(n - 1)q$
 - Timer interrupt is needed to check on quantum expiry
 - Choice of time quantum is important as it affects CPU utilisation and waiting time
- Priority Scheduling
 - Some processes are more important than others, assign priority values to them, select highest priority task
 - Can be preemptive where higher priority processes can jump in or non-preemptive where late tasks have to wait
 - Low priority process might starve if CPU is hogged
 - Can be solved by decreasing priority of currently running process after each time quantum, or give current running process a time quantum
 - Priority Inversion: Lower priority task ends up blocking higher priority task

when medium priority task keeps blocking it, and they share resources

- Multi Level Feedback Queue (MLFQ)
 - Adaptive, learns process behaviour, minimises response and turnaround time
- Rules
 - If $p_a > p_b$, a runs
 - If $p_a = p_b$, a and b run in RR
 - New job has highest priority
 - If a job fully utilises its time slice, its priority is reduced
 - If a job gives up or blocks, its priority is retained
- Lottery Scheduling
 - Give out lottery tickets to processes for various system resources, winner is granted the resource
 - Responsive, and newly created process can participate in the next lottery
 - Provides level of control; tickets can be distributed to child processes, important processes can be given more tickets

Cases from Tutorial

- Lengthy CPU phase followed by IO intensive phase: Priority can sink during CPU intensive phase, so it might not get enough in the next phase; Solve by moving all process in the system to the highest priority periodically
- Process repeatedly gives up CPU just before time quantum, retains priority indefinitely; Solve by accumulating CPU usage across time quanta

Inter-Process Communication

- Allow cooperating processes to share information
- Shared Memory
 - First process creates shared memory region
 - Second process attaches this to its own memory space
 - Processes can now communicate using this memory region
 - OS is only involved in creation and attaching
 - Efficient and provides ease of use
 - Synchronisation of access is needed and implementation might be harder
 - For POSIX SHM in *nix, need to destroy after use; only if not attached to any process

```
int shmId = shmget(IPC_PRIVATE, 40, IPC_CREAT | 0600);
int *shm = (int*) shmatt(shmId, NULL, 0);
shmdt((char*) shm);
shmctl(shmId, IPC_RMID, 0);
```

- Message Passing
 - First process prepares a message and sends it to another process
 - Second process can receive and process this message, usually provided as syscalls
 - Additional properties like naming and synchronisation are needed
 - Message has to be stored in kernel memory space, operations require OS
 - Naming Scheme: Direct Communication: Sender or receiver explicitly names other party, one link per pair of communicating processes e.g. Unix domain socket (filepath)
 - Naming Scheme: Indirect Communication: Messages sent or received from message storage known as mailbox or port, can be shared across multiple processes e.g. Unix message queue
 - Synchronisation Behaviours: Blocking Primitives: Synchronous, process is blocked until message is sent or received
 - Synchronisation Behaviours: Non-Blocking Primitives: Asynchronous, sender resumes immediately, receiver does not wait and gets indication of status
 - Portable and easy to synchronise
 - Inefficient and harder to use

- Unix Pipes
 - Unix processes have 3 default communication channels: stdin, stdout, stderr
 - Pipes can be used to link input and output channels to one another, shared between two processes, data is FIFO
 - Pipe is a circular bounded byte buffer with implicit synchronisation, writer waits if full, reader waits if empty
 - Variants can have multiple readers and writers
 - Pipes can be half-duplex: Unidirectional, one write and one read end, or full-duplex: Bidirectional, both read and write
 - For pipe(), array of file descriptors is returned with reading and writing end along with 0 for success status
 - Can pipe standard communication channels, use dup() or dup2()

```
int pipeFd[2];
pipe(pipeFd);
write(pipeFd[WRITE_END], str, strlen(str)+1);
int len = read(pipeFd[READ_END], buf, sizeof(buf));
close(pipeFd[READ_END]);
// close end not being used
```

- Unix Signal
 - Asynchronous notification regarding an event sent to a process or thread
 - Recipient must handle signal using a default set of handlers or user supplied handler
 - UNIX examples include Kill, Interrupt, Memory Error etc.
 - signal(SIGSEGV, handler), void handler(int signo)

Threads

- Processes are expensive to create and context switch between
- Communication also relies on IPC
- Each process starts with a single thread of control, and more can be added
- Threads share memory context (text, data, heap) and OS context (PID and other resources)
- Identified by thread id, and have unique registers and stack
- Thread switching is more lightweight as only register and stack needs to be swapped (hardware context) v/s process context switch (OS, hardware, memory context)
- Less resources are needed to manage, and no need for IPC, can also take advantage of multiple CPUs
- However, need to guarantee correctness and determine correct behaviour of parallel execution
- Affects process operations

Thread Models

- User Thread: Implemented as a user library, runtime system in process will handle thread related operations, kernel is not aware of threads in process
- Can have multithreaded program on any OS, generally more configurable and flexible
- Since scheduling is performed at process level, whole process gets blocked at once, cannot exploit multiple CPUs
- Kernel Thread: Implemented in OS, handled as system calls, and thread-level scheduling is possible
- More than 1 thread in the same process can run at the same time on multiple CPUs
- Thread operations are slower and more resource intensive as system calls
- Generally less flexible, and might be expensive or overkill
- Hybrid Thread Model: Have both, OS schedules kernel threads only, user thread can bind to kernel thread, offers greater flexibility
- Threads started as a software mechanism, but have hardware support on modern processors in the form of multiple sets of registers allowing them to run

in parallel on the same core, known as simultaneous multi-threading (SMT)

POSIX Threads

- Standard defined by IEEE, but implementation is not specified
- pthread library, with methods like create, exit, join

```
int pthread_create(
    pthread_t* tidCreated,
    const pthread_attr_t* threadAttributes,
    void* (*startRoutine) (void*),
    void* argForStartRoutine);
int pthread_exit(void* exitValue);
int pthread_join(pthread_t threadID, void **status);
```

Synchronisation

- When multiple processes share resources, can cause synchronisation problems
- Execution of single process is deterministic, while execution of concurrent process may be non-deterministic due to race conditions involving modifying / access of the same variable
- Designate code segment with race condition as critical section where only one process can execute in there at a time
- Properties
 - Mutual Exclusion: If a process is executing in the critical section, all other process are prevented from entering
 - Progress: If no process is in a critical section, one of the waiting process should be granted access
 - Bounded Wait: After a process requests to enter critical section, there exists an upper bound of number of times other processors can enter before it
 - Independence: Process not executing in critical section should never block other processes
- Incorrect Synchronisation can cause
 - Deadlock: All processes are blocked and no progress is made
 - Livelock: Processes keep changing state to avoid deadlock and make no other progress but are not blocked
 - Starvation: Some processes never get to make progress because they are perpetually denied necessary resources
- Can have assembly / HLL implementations or high level abstraction
- Assembly Level Implementation
 - TestAndSet: Atomic instruction (single machine operation) provided by processor to aid synchronisation
 - Loads content from memory location into register, stores 1 into memory location
 - Performed as a single machine operation
 - Example Usage:

```
void EnterCS(int* Lock) {
    while(TestAndSet(Lock) == 1);
}
void ExitCS(int* Lock) {
    *Lock = 0;
}
```

- Works, but uses busy waiting
- Variants like Compare and Exchange, Atomic Swap, Load Link / Store Conditional exist
- High Level Language Implementation
 - Peterson's Algorithm

```
Want[0] = 1;
Turn = 1;
```

```

while(Want[1] && Turn == 1);
// CS
Want[0] = 0;

Want[1] = 1;
Turn = 0;
while (Want[0] && Turn == 0);
// CS
Want[1] = 0;

```

- Assumes updating Turn is atomic
- Causes busy waiting due to repeated while loop
- Lower level so potentially more error prone, also not very general
- High Level Abstraction
 - Semaphore: General synchronisation mechanism; only behaviour is specified rather than implementation
 - Provides a way to block / sleep processes or unblock / wake them up
 - Semaphore contains an integer value and has two atomic semaphore operations, "counts how many resources are available"
 - Wait() blocks if not positive, and decrements
 - Signal() increments, and wakes up sleeping processes if any, never blocks
 - Invariant: $S_{current} = S_{initial} + signal(S) - wait(S)$ where it refers to number of signal() executed and wait() completed
 - Can have general or binary semaphores, with general semaphores used for convenience
 - Example: Wait before CS, Signal after completion, known as mutex
 - Deadlock is still possible with incorrect use of semaphores e.g. multiple overlapping semaphores
 - Common Alternative: Conditional variable, allow task to wait for certain event to happen, has ability to broadcast (wake up all waiting tasks), related to monitor

Classic Synchronisation Problems

- Producer Consumer
 - Processes share a bounded buffer of size K , producers insert items in buffer, consumers remove items from buffer

```

// busy waiting
count = in = out = 0
mutex = S(1)
cP = TRUE
cC = False
// producer
while (TRUE) {
    while(!canProduce);
    wait(mutex);
    if (count < K){
        // remove item
        canConsume = TRUE;
    } else {
        canProduce = FALSE;
    }
    signal(mutex);
}
// consumer
while (TRUE) {
    while(!canConsume);
    wait(mutex);
    if (count > 0) {
        // insert item

```

```

        canProduce = TRUE;
    } else {
        canConsume = FALSE;
    }
    signal(mutex);
}

// blocking version
count = in = out = 0
mutex = S(1)
notFull = S(K)
notEmpty = S(0)
// producer
while (TRUE) {
    wait(notFull);
    wait(mutex);
    // remove item
    signal(mutex);
    signal(notEmpty);
}
// consumer
while (TRUE) {
    wait(notEmpty);
    wait(mutex);
    // insert item
    signal(mutex);
    signal(notFull);
}

```

- Similarly for consumer process
- Similar logic, but allows processes to go to sleep to save processing power
- Readers Writers

```

// writer
while (TRUE) {
    wait(roomEmpty);
    // modify
    signal(roomEmpty);
}
// reader
while (TRUE) {
    wait(mutex);
    nReader++;
    if (nReader == 1)
        wait(roomEmpty);
    signal(mutex);
    // read
    wait(mutex);
    nReader--;
    if (nReader == 0)
        signal(roomEmpty);
    signal(mutex);
}

```

- Processes share a data structure, writers must have exclusive access, reader can access with other readers
- Writer waits for room to be empty, then writes
- Reader waits for mutex, then waits for room empty, before reading then signalling
- Dining Philosophers

- Each philosopher has a left and right chopstick, and needs both to eat
- Tanenbaum Solution

```

// mutex = 1;
void philosopher(int i) {
    while (TRUE) {
        think();
        takeChpStcks(i);
        eat();
        putChpStcks(i);
    }
}
void takeChpStcks(i) {
    wait(mutex);
    state[i] = HUNGRY;
    safeToEat(i);
    signal(mutex);
    wait(s[i]);
}
void safeToEat(i) {
    if (state[i] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING) {
        state[i] = EATING;
        signal(s[i]);
    }
}
void putChpStcks(i) {
    wait(mutex);
    state[i] = THINKING;
    safeToEat(LEFT);
    safeToEat(RIGHT);
    signal(mutex);
}

```

- With an empty seat, deadlock is impossible

Synchronisation Implementations

- POSIX Semaphore: wait() and signal() implemented
- pthread has mutex as a binary semaphore with lock and unlock
- Also has conditional variables with wait, signal, and broadcast
- Most HLL have some level of thread and synchronisation support